

DOCUMENTO DE MODELAGEM DE SOFTWARE

SISTEMA TECHCITY CONTROL – MÓDULO 2: SISTEMA DE ENTREGAS COM DRONES

O presente documento constitui o artefato oficial de especificação e modelagem para a primeira entrega do Módulo 2 (Sistema de Entregas com Drones) integrante da plataforma unificada TechCity Control. Este sistema foi concebido para atender às demandas de modernização urbana da prefeitura de TechCity, fornecendo uma infraestrutura computacional robusta para o gerenciamento de uma frota de Veículos Aéreos Não Tripulados (VANTs), comumente denominados drones, aplicados à logística de entregas rápidas e prioritárias.

1. Descrição Geral do Sistema

O Módulo 2 do TechCity Control opera como uma camada especializada de automação logística em memória, abstendo-se do uso de frameworks robustos ou bancos de dados relacionais nesta etapa, em estrita conformidade com as restrições pedagógicas estabelecidas. Sua arquitetura fundamenta-se nos princípios puros da Programação Orientada a Objetos (POO), empregando o encapsulamento estrito para mitigar efeitos colaterais e garantir a integridade dos dados operacionais da cidade.

O fluxo operacional compreende a inserção de novos drones à frota pública e a recepção contínua de demandas de frete urbanas. O núcleo de inteligência reside no controlador central, o qual desempenha o papel de mediador de alta coesão e baixo acoplamento (Padrão Controller/Fachada). Ele valida parâmetros críticos, tais como a capacidade física de carga útil dos dispositivos antes de autorizar qualquer despacho, e instancia a entidade associativa de controle denominada "Entrega" para o monitoramento em tempo real do ciclo de vida logístico. Para garantir a conformidade institucional e a governança de segurança urbana, o módulo integra-se ao barramento do Core unificado da prefeitura. Todas as transições de estado críticas (como a decolagem, cancelamento ou conclusão de rotas) exigem a validação de um operador humano previamente autenticado e ativo no sistema, gerando instantaneamente registros cronológicos e imutáveis de auditoria (Logs).

2. Lista de Requisitos Funcionais

2.1. Núcleo Comum Obrigatório (Core)

- **RF01 – Cadastro de Usuários:** O sistema deve permitir o registro formal de operadores da plataforma, capturando obrigatoriamente um identificador alfanumérico exclusivo (ID), o nome completo do operador e o flag booleano indicativo do seu estado de ativação.
- **RF02 – Controle de Usuários Ativos:** O controlador deve expor métodos específicos para modificar o estado de ativação de qualquer usuário. Usuários marcados como inativos ficam sumariamente impedidos de assinar ou executar qualquer rotina operacional no ecossistema municipal.
- **RF03 – Registro de Operações (Log de Auditoria):** O sistema deve forçar a geração de um registro histórico irrefutável e imutável para cada transição de negócio relevante, arquivando uma descrição detalhada do evento, o carimbo de data/hora (timestamp) no formato ISO e a associação ao usuário responsável pela ação.
- **RF04 – Controlador Principal:** Implementação de uma classe controladora unificada responsável por deter as referências das coleções em memória, coordenar os fluxos de caso de uso e servir de fachada exclusiva para as interações externas do sistema.

2.2. Módulo 2 – Requisitos Obrigatórios

- **RF05 (M2-RF01) – Cadastro de Drones:** O sistema deve permitir a expansão da frota através do registro de novos dispositivos voadores, armazenando seu código identificador único, sua capacidade de carga em massa (kg) e iniciando seu estado físico como "Disponível".
- **RF06 (M2-RF02) – Registro de Pedidos de Entrega:** O sistema deve viabilizar a entrada de ordens de serviço contendo uma descrição dos itens, o destino geográfico da entrega, o peso líquido da encomenda (kg) e inicializando o status de ciclo de vida como "Pendente".
- **RF07 (M2-RF03) – Associação de Drones às Entregas:** O controlador deve orquestrar o vínculo entre um drone "Disponível" e um pedido "Pendente", instanciando o objeto "Entrega" com status de voo definido como "Iniciada", alterando simultaneamente o status do drone para "Em Operação" e o do pedido para "Em Rota".
- **RF08 (M2-RF04) – Atualização do Ciclo de Vida das Entregas:** O sistema deve permitir que as entregas com status "Iniciada" tenham sua conclusão decretada. Essa operação altera o status da Entrega para "Concluída", o status do Pedido para "Entregue" e devolve o Drone associado ao status "Disponível", reinserindo-o de forma imediata na malha logística ativa.

2.3. Requisitos Adicionais Propostos (Desenvolvidos pela Equipe)

- **RF09 (Adicional) – Validação Automatizada de Sobrecarga de Carga Útil:**
Descrição: Previamente à efetivação do acoplamento logístico estipulado no RF07, o controlador central deve interceptar a requisição e comparar a massa líquida descrita no

objeto Pedido com a capacidade máxima de sustentação parametrizada no objeto Drone. Se o peso do pedido ultrapassar o limite suportado pela aeronave, a operação de despacho deve ser vetada, um erro de violação operacional deve ser emitido e nenhuma mudança de estado deve ser consolidada.

Justificativa: Este requisito mitiga riscos severos associados à segurança aeroespacial urbana e à integridade do patrimônio municipal. Drones operando acima do limite nominal de carga sofrem superaquecimento de motores, esgotamento precoce de baterias e severa perda de estabilidade aerodinâmica, apresentando risco crítico de queda em áreas habitadas.

- **RF10 (Adicional) – Cancelamento de Entrega com Despacho de Retorno e Liberação Imediata:**

Descrição: O sistema deve dispor de uma rotina supervisionada para abortar missões logísticas ativas (status "Iniciada"). O acionamento desta operação modificará o status da entidade Entrega para "Cancelada", atualizará o status do Pedido para "Cancelado" e reverterá instantaneamente o estado do Drone para "Disponível".

Justificativa: Ambientes urbanos reais impõem volatilidade operacional imprevista (alertas meteorológicos súbitos, fechamento de espaço aéreo por órgãos reguladores ou desistência do cidadão solicitante). O sistema necessita de flexibilidade e resiliência para abortar trajetórias de forma segura, reinserindo o dispositivo móvel na frota disponível sem causar travamento de recursos ou inconsistência lógica nos indicadores governamentais.

3. Lista de Classes e Responsabilidades (Dicionário de Classes)

A tabela a seguir consolida as classes de domínio projetadas para o sistema, detalhando seus atributos estritamente protegidos e delimitando suas responsabilidades em conformidade com os princípios da alta coesão.

Nome da Classe	Atributos Encapsulados (#)	Responsabilidade Principal
Usuario	id (String), nome (String), ativo (Boolean)	Abstrair os recursos humanos da prefeitura, controlando individualmente sua aptidão jurídica e permissiva para operacionalizar o sistema da cidade inteligente.

Nome da Classe	Atributos Encapsulados (#)	Responsabilidade Principal
Operacao	descricao (String), dataHora (Date), usuario (Usuario)	Materializar uma entrada imutável e perene de auditoria histórica, acoplando a descrição semântica da ação realizada ao seu respectivo autor e data.
Drone	id (String), capacidadeCarga (Number), status (String)	Representar as restrições físicas e dinâmicas da unidade de voo autônomo, gerenciando autonomamente suas transições operacionais ("Disponível", "Em Operação", "Manutenção").
Pedido	id (String), descricao (String), destino (String), peso (Number), status (String)	Encapsular as necessidades de transporte urbano, salvaguardando dados da carga e gerenciando seu respectivo progresso técnico de atendimento ("Pendente", "Em Rota", "Entregue", "Cancelado").
Entrega	id (String), drone (Drone), pedido (Pedido), status (String)	Funcionar como classe de associação complexa que consolida a missão logística ativa, amarrando de forma unívoca a aeronave responsável à carga transportada.
TechCityController	usuarios (Array), drones (Array), pedidos (Array), entregas (Array), operacoes (Array)	Centralizar o barramento lógico do sistema. Atua como o cérebro organizacional da arquitetura, interceptando

Nome da Classe	Atributos Encapsulados (#)	Responsabilidade Principal
		comandos externos, aplicando regras interclasses, realizando validações e provendo persistência volátil em memória.

4. Relacionamentos entre Classes e Mapa de Arquitetura

A malha arquitetural do projeto estrutura-se de forma estritamente horizontal, abdicando de vínculos de herança e especialização hierárquica, privilegiando a flexibilidade dada por acoplamentos por delegação e composição:

- **Agregação por Referência Permanente:** A classe de controle principal (TechCityController) estabelece um vínculo de agregação de cardinalidade 1:N com todos os elementos de domínio. As listas internas guardam ponteiros para os objetos em memória, possuindo ciclos de vida independentes, o que permite que as entidades persistam logicamente mesmo com reinicializações patrimoniais do controlador.
- **Associação Direcionada de Auditoria:** A classe Operacao retém um acoplamento unidirecional de cardinalidade 1:1 com a classe Usuario. Isto significa que a instância de Log conhece intrinsecamente a identidade do operador que disparou o evento, permitindo a rastreabilidade retroativa sem violar o isolamento das responsabilidades do usuário.
- **Acoplamento Associativo Estrutural:** A classe Entrega personifica o cruzamento relacional entre o motor de transporte (Drone) e o objeto físico de valor (Pedido). Ela detém simultaneamente referências diretas 1:1 para ambas as entidades, atuando como o elo de transição que governa o andamento logístico unificado.

5. Considerações de Encapsulamento de Dados

Em estrito cumprimento aos padrões de qualidade de Programação Orientada a Objetos, todas as variáveis de instância pertencentes às classes deste módulo foram declaradas sob escopo privado imutável por meio do operador nativo do JavaScript (sintaxe do prefixo #).

Esta barreira física impede alterações acidentais ou indevidas a partir do escopo externo. A

comunicação com o estado interno ocorre unicamente sob rígida mediação semântica:

- **Getters Explícitos e Seguros:** Métodos seletores que não expõem referências mutáveis de propriedades sensíveis sem necessidade (ex: `getCapacidadeCarga()`, `getPeso()`).
- **Setters com Restrição de Negócio:** Exclusão completa de modificadores abertos e genéricos. As alterações de propriedades vitais são acionadas por métodos expressivos (ex: `setStatus()`), os quais respondem ao fluxo orquestrado e validado unicamente pelo `TechCityController`.